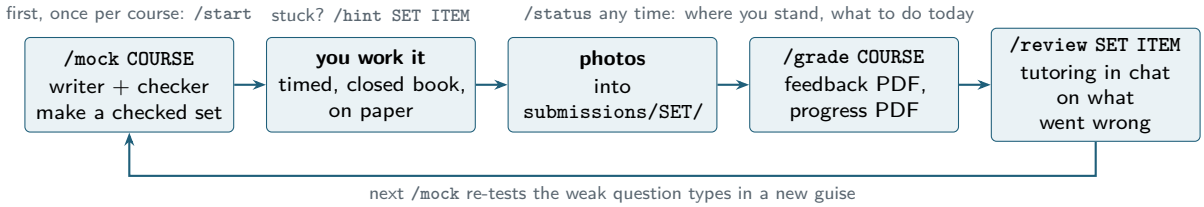
Exam prep: how to use it

Practice sets in the style of your real finals, graded from photos of your work, each one aimed at what went wrong in the last.

The loop



`COURSE` is the short key you gave the course in **`/start`** (`MATH`, say); `courses.md` lists them. Plain requests work too: "make the next MATH set", "the photos of MATH-T01 are in", "how am I doing?", "give me a lesson on line integrals".

Getting started

1. Install what the project needs (Python with sympy, TeX Live, ImageMagick, poppler, git): `SETUP.md`, about 20 minutes, mostly waiting. On Windows the project runs inside WSL, the Linux built into Windows. `python3 tools/doctor.py` checks everything and says what is missing.

2. Set up the desktop app (Code tab):

Permissions	*Bypass permissions* in the mode selector of this folder, after allowing it under Settings → Claude Code. Every command runs scripts and agents; without it you approve dozens of steps per set. Claude then acts without asking, so keep it to this project. *Auto* mode is the cautious alternative.
Model	Opus for the main session: it plans and hands the work to cheaper agents.
Notifications	On. Writing a set takes 10–30 minutes, grading 5–10.
Windows	Start sessions in the *WSL* environment (Ubuntu). Your files: in Explorer at `\\wsl.localhost\Ubuntu\home\...`; pin that folder.
Sessions	A new session for each task (a set, a grading, a review). Long threads re-read their history on every message.

3. Type `/start`. It checks the installation, asks per course for the name, the date of the final, the other parts of the grade (midterm, homework, lab) and the pass rules, and your study days. Then it tells you where to put the course's files, `<course>/materials/`, most useful first:

- **past exams**, as many as you can find, each with its answers or solutions (three or more make a good course; with fewer, sets match the real exam less well);
- the **study guide** with the exam rules, and the **lecture schedule** with dates;
- the **formula sheet** allowed in the exam;
- **lecture notes**, slides or book chapters in scope, and **exercise sets** with solutions;
- for a Mathematica part: the course's notebooks.

Any file names, any sub-folders. Tell **`/start`** which past exams you have already worked.

It then builds the course, one agent at a time: it sorts your files, proposes one real final to **hold out** unseen (you decide), transcribes the other exams into a question bank, writes the syllabus and formula sheet, collects course exercises for topics the old exams miss, and analyses everything into a **blueprint**: the question types of your exam, their weights, how they vary, and where students lose points. Finally it works out the grade your final must reach, and writes a day plan into `courses.md`. Building a course costs roughly one 5-hour usage window on a Pro plan; it stops and resumes cleanly, so run **`/start`** again to continue. Then **`/mock COURSE`** makes the first, diagnostic set.

The six commands

`/start [add|new-quarter]`

Set-up, explained above. Run it again at any time: it continues where it stopped. `add` adds a course; `new-quarter` moves the old courses to `archive/` (nothing is deleted) and sets up the new ones.

/mock COURSE [diagnostic|train|full|holdout] [MINUTES]

Makes the next practice set and sends you `exam.pdf`. The key stays hidden until the set is graded.

- `diagnostic`: every question type once, at exam level. `train`: a targeted set on your weakest types. `full`: a complete mock in the real exam's format and time. `holdout`: the reserved real final (benchmark, plan day only or on request).
- No kind given: `diagnostic` if nothing is graded yet, otherwise today's entry in the plan, otherwise `train`.
- `MINUTES`: time budget, training sets only, default 90.
- If an earlier set is still ungraded, it asks you to grade or drop it first. Say so if you will sit the set on a later day, so topics not yet lectured are judged for that day.

Examples: `/mock MATH` `/mock MATH train 60` `/mock MATH full`

/hint [SET] ITEM

While you work a set: the smallest hint that gets you started on one item, never the solution.

- It first asks what you have so far. Then one rung per ask: **nudge** (a question pointing at what in the problem gives the approach away), **method** (named, and why it applies), **set-up** (the first step written down, not worked out). Nothing more until it is graded: no answers, no checking your result.
- Every hint is logged in `submissions/SET/hints.json`. On paper: a circled **H** where you continue, and pause the timer while you chat.
- At grading, work above the H scores as usual. What the hint gave and the lines built on it score nothing, and you also see what the answer would have scored with the hint allowed.
- The next training set brings that question type back *first*, with the same move behind a different cue. `progress.pdf` lists each hint and how the re-test went.
- Training sets mainly. On a diagnostic or full mock it warns first; none on the held-out exam.

Examples: `/hint MATH-T01 5.2` `/hint 3.1` (the set you are working on)

/grade COURSE [SET]

Grades your handed-in set, records the scores and rebuilds the progress report.

- `SET`: default is the newest set without feedback.
- Before you run it, put photos of every page, in order, in `<course>/submissions/SET/` (jpg, png, heic, webp or pdf). Multiple-choice letters, time spent and skipped items go in `answers.txt` there or in chat. Mathematica work goes in as the `.nb` file. Photos still in Downloads? Say so; it copies them.
- It reports the page count. Check it, because a missing page is graded as blank.
- You get the feedback PDF, the key PDF and `progress.pdf`. In chat: the score and grade equivalent, the top two or three fixes, any readings to double-check, and the items worth a `/review`.
- Think a reading or deduction is wrong? Say which item and why. It gets regraded, and the old rows are replaced.
- It never starts the next set. That waits for your `/mock`.

Examples: `/grade MATH` `/grade MATH MATH-D01`

/review SET ITEM

A tutoring thread on one graded question: what you wrote, where and why it went wrong, how to do it right.

- `/review MATH-D01 3.2`. With only `SET`, or `latest`, it lists the items and suggests the ones that lost the most points. Loose names work too ("the flux one").
- It opens with a diagnosis: the line where it went wrong, the misunderstanding behind it, and the right first move. Then you choose **guided** (hints, you write each step) or **explained** (the full solution, with checks you can use in the exam).
- Ask follow-ups in the same thread. It ends with you stating the method in your own words, a redo from a blank page, and a fresh variant to try.
- Ask for a **video** ("animate it") and you get a 1–2 minute clip: your own lines appear one by one, the wrong term is boxed with the reason, then it rewinds to your last good line and continues correctly (saved in `<course>/animations/`). Every line is numbered: ask "why line 12?" and the answer comes in chat. Needs the optional video packages.
- If a whole question type is missing, it offers a **lesson** PDF (saved in `<course>/lessons/`). Scores are never changed here.

/status

Rebuilds and sends `progress/progress.pdf`. For each course it gives the days and study days left, the sets done, the grade figures (with the share of the exam they are measured on) and any ungraded set. It also says what to do today: the earliest exam first, then the weakest heavy question types.

Handing in: the conventions

- **Timed and closed book**, with only the aids on the cover. Otherwise the score means nothing.
- Items marked *not yet lectured*: write `SKIP`. They are not counted.
- **Stuck before you can start?** Use **`/hint`**: write a circled H where you continue, pause the timer. The log tells the grader the rest.
- **Hint from an AI tutor elsewhere?** Draw a thick black bar in the margin next to the lines written with help, and say what the hint was about. Those lines score nothing, and that question type will not move up a level.
- **Time stamps** (elapsed time) under each problem. An answer that is correct but took more than 1.5 times its budget does not move up a level either.
- **Photos in page order, into the folder**, not pasted into the chat (the grader cannot see pasted images, and the main session pays to read them). `prep_photos.sh` sorts them by capture time, then by file name, rotates and shrinks them into `pages/`, and never touches the originals.

Ground rules

- Sets are written **only on your command** (**`/mock`**). Grading never starts one.
- No set reaches you before the checker passes it, and no key before your work is graded.
- One real final per course is **held out** in `<course>/holdout/`: no agent that writes or checks sets ever opens it, and you sit it as `-H01` in the last week for an honest benchmark.
- Scores come only from the grader's JSON via `ledger.py record`, never from editing by hand.
- One agent at a time, earliest exam first, to stay under the usage limit.
- Your files, work and grades stay on your computer, and git ignores them.

Where things are

`courses.md`	your courses, exam dates, held-out exams, banked grades and the day plan
`<course>/materials/`	the files you dropped in; `intake.json` says what each one is
`<course>/holdout/`	the reserved real final, unseen until you sit it
`<course>/bank/`	the past exams, transcribed; `blueprint.md` next to it is the analysis
`<course>/sets/SET/`	`exam.pdf`, `key.pdf`, `spec.json` (why these items), `set.json`, `check/` (checker's verdict)
`<course>/submissions/SET/`	your photos, `answers.txt`, notebooks, `hints.json` (from **`/hint`**); `pages/` is generated
`<course>/feedback/SET.pdf`	the grader's feedback for that set
`<course>/lessons/`	lesson PDFs, one per question type you asked about
`<course>/ledger.jsonl`	every graded item ever: the source for all statistics
`progress/progress.pdf`	the one progress report for all courses
`archive/`	the courses of earlier quarters

Set ids: `MATH-D01` diagnostic, `-T01, -T02, ...` training, `-F01` full mock, `-H01` held-out real exam.

Under the hood

Step	*Who*	What happens
Set-up	*Sonnet agents, Opus*	`intake.md` sorts your files, `transcribe.md` turns each past exam into the *bank*, `syllabus.md`, `seeds.md` and `toolkit.md` write the course facts, conventions and extra exercises, and an Opus agent writes the *blueprint* (`blueprint.md`). `course.py` keeps track of the steps.
Spec	*`ledger.py next`*	Picks the question types ("archetypes"), levels, points and minutes from your ledger. No AI involved.
Write	*Opus agent*	`prompts/writer.md`: writes the set and key from the spec, the blueprint and the bank, avoiding variations you have already seen.
Check	*Sonnet agent*	`prompts/checker.md`: solves the set *blind*, then compares with the key. If it says `FIX NEEDED`, the writer fixes the set (at most two rounds), and an item that still fails is dropped. `ledger.py checkset` then validates the files.
Grade	*Sonnet agent*	`prompts/grader.md`: transcribes your handwriting, grades against the rubric, tags the first error, and writes feedback as `.json` and `.pdf`.
Record	*`ledger.py record`*	Validates the grader's JSON into `ledger.jsonl`, the only source of scores. `build_report.py` then writes `progress.pdf`.
Hint	*`hints.py`*	Loads one item of a set you are working (question, earlier hints, and the key for the tutor's eyes only) and logs each hint. `ledger.py record` refuses feedback that credits work done after a logged hint.
Review	*`item_context.py`*	Loads one item (question, your work, grading, key, pitfalls) into chat. `select_review.py` ranks the items worth reviewing. For a video the session writes a storyboard (`prompts/storyboard.md`) and `animate.sh` renders it with Manim.
Extras	*Sonnet agents*	`lesson.md`: a mini-course on one archetype. `holdout.md`: turns the reserved real exam into a gradable set without changing it.

The chat session is only the *orchestrator*. It runs scripts and passes files between agents, and never reads exam PDFs, photos or keys itself. That keeps each session cheap and long-lived.

How the next set adapts to you

- **Mastery** per question type is a recency-weighted average, where each older attempt counts half as much. Each attempt adds the evidence $\min(1,\ \text{score fraction} \times c)$, with $c = 0.65,\ 1.0,\ 1.1$ for levels 1, 2, 3. Skipped items are ignored and blanks count as 0.
- **Status:** *exam-ready* (≥ 0.80 and last tried at level ≥ 2), *close* (≥ 0.55), *needs-work*, *unmeasured*.
- **Next level** for a question type: $\geq 80\%$ goes one level up, unless you had help or took more than 1.5 times the budget. 50–80% stays the same. Below 50% goes one level down, except after help when the rest of the answer was right ($\geq 80\%$ with the help allowed): then it stays.
- **Training sets** rank the lectured question types by weight $\times$ (1 − mastery), $\times 1.5$ if the last attempt was below 50%, plus a small bonus for staleness, and fill the minute budget from the top. They also add one *retention check* on an exam-ready type you have not seen for 5 or more days. Types you needed a **`/hint`** for last time come first. The writer sees your recent errors on each type, and for a hint what you had to be told and what should have cued it.
- **Grade figures** (out of 10): *on measured part* is the grade over the types tried so far; *coverage* is the share of the exam's weight that has been measured; *banked* counts untried types as 0, a lower bound. The pass line in the report is the grade your final must reach, given your other components.
- **Error tags:** concept, method, setup, algebra, sign, arithmetic, notation, justification, misread, incomplete. "Where the points go" in `progress.pdf` totals them.

What it costs

Claude plans meter usage per 5-hour window and per week, in percent, not dollars. To compare tasks, the table gives the *API-equivalent*: the tokens priced at list rates (Opus $4 in / $20 out, Sonnet $2 / $10 per million tokens; re-reading cached context $0.20, writing it 1.25× the input price). **Fresh** tokens are new input plus output; **re-read** tokens are the context a model sends again on every step, at 5–10% of the fresh rate.

Function	Who works	Fresh	Re-read	$	of a Pro 5 h window
`/start, one course` *	Sonnet agents, Opus blueprint	1–2M	15–30M	15–30	60–120%
`/mock train`	Opus writer, Sonnet checker	0.8M	8M	6.7	26%
`/mock diagnostic`	same	0.9–1.7M	11–23M	8–14	31–54%
`/mock full` *	same, larger set	∼1.2M	∼12M	10–16	38–62%
`holdout prep` *	Sonnet	∼0.3M	∼4M	1–3	4–12%
`/grade`	Sonnet grader, chat	0.3M	2.2M	1.3–1.5	5–6%
`/hint, one item` *	chat session	∼20k	∼0.4M	0.2–0.5	1–2%
`/review, 10 turns` *	chat session	∼0.2M	∼3M	1–1.5	4–6%
`+ video` *	storyboard, render is free	∼30k	∼0.5M	+0.5	+2%
`lesson`	Sonnet	0.16M	4.8M	1.7	7%
`/status` *	scripts, one turn	<20k	0.3M	0.1–0.3	<1%
`one chat message`	warm cache	∼3k	0.1M	0.03–0.10	0.1–0.4%
`new session, 1st message`	cache is written	0.11M	—	0.3–0.6	1–2%

Rows without * were measured on real runs in October 2026; rows with * are estimates from the nearest measured run. The last column uses one calibration on a Pro plan: about $26 API-equivalent filled a 5-hour window and about $200 a week (±30%; Anthropic does not publish the limits). Max plans have several times more.

- **The 5-hour window is the one that bites.** On Pro, about three training sets or two full mocks fill it. Writing a set is ∼60% of the cost of a loop, so space the writing out over the day, and set up one course per window.
- **A heavy study day** (two training sets, four gradings, eight reviews) is ∼$29 API-equivalent: about one Pro window.
- **Cheaper chat:** a review thread on Sonnet costs half per message. Start a new session for a new task.
- **Extra usage** (credits), if you turn it on, bills work past a limit at API rates, so the dollar column is roughly what you would pay for the part that overflows.
- **Calibrate for your plan:** note the app's usage meters before and after a task and compare with `python3 tools/usage_report.py`, which shows what this project used per day and model. Chats outside this project use the same limits and are not counted there.

By hand, if you want

```
python3 tools/doctor.py                                  is everything installed?
python3 tools/course.py list                             courses and how far their set-up is
python3 tools/course.py state MATH                       the set-up steps of one course
python3 tools/ledger.py status                           all courses at a glance
python3 tools/ledger.py mastery MATH                     mastery table for one course
python3 tools/ledger.py next MATH --kind train --minutes 60   preview what a set would target
python3 tools/build_report.py                            rebuild progress.pdf
python3 tools/item_context.py MATH-D01                   list a set's items and scores
python3 tools/hints.py show MATH-T01                     the hints logged for a set
python3 tools/usage_report.py                            what this project used, per day and model
```